Assignment 3
Distributed K-Nearest Neighbors (KNN) with MPI

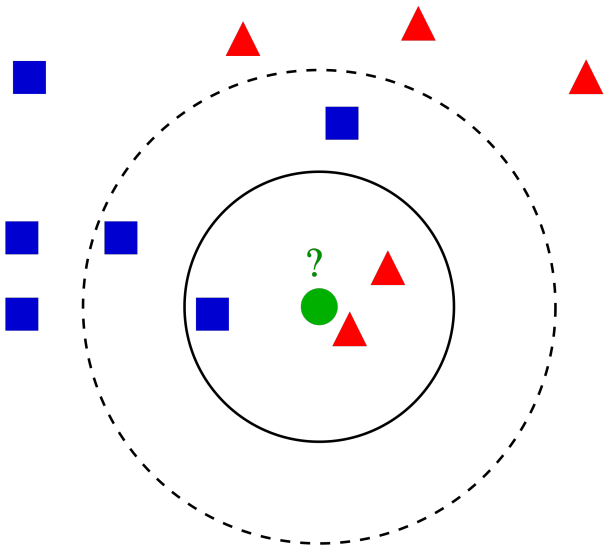
CS3210 – 2025/26 Semester 2

Learning Outcomes

This assignment allows you to apply your understanding of distributed-memory parallel programming with MPI to design and implement a distributed K-Nearest Neighbors (KNN) algorithm. You will learn how to manage data distribution, inter-process communication, and synchronization, as well as measure performance scaling as the number of MPI processes increases.

1 Problem Scenario

In this assignment, you will implement the K-Nearest Neighbors (KNN) algorithm using MPI. KNN is a classic example of a computation that is highly parallelizable — each query can independently compute distances to all data points. Your task is to parallelize this process efficiently across multiple MPI processes running on different nodes. Your implementation should be correct, scalable, and demonstrate improved performance with increasing numbers of MPI processes.



Source: https://commons.wikimedia.org/wiki/File:KnnClassification.svg

2 Overview of the K-Nearest Neighbors Algorithm

The **K-Nearest Neighbors (KNN)** algorithm is one of the simplest and most intuitive methods in machine learning for classification and regression tasks. It operates under a single key principle: *similar data points exist close to each other in feature space*. Despite its simplicity, KNN is widely used in practice for tasks such as image recognition, recommendation systems, and anomaly detection. It is also often employed as a baseline in machine learning pipelines because of its interpretability and general applicability to many types of data.

2.1 How KNN Works

Given:

- A set of labeled training data points $\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$ where each x_i is a feature vector and y_i is its label.
- A new unlabeled query point q.

The KNN algorithm proceeds as follows:

1. Compute the Euclidean distance from q to every data point x_i in the dataset.
2. Identify the k data points with the smallest distances — these are the *nearest neighbors*.
3. For classification, assign q the most frequent label among its k nearest neighbors.

Note: To simplify calculations and avoid precision issues for the following assignment, we will calculate the square of the Euclidean distance.

2.2 Input and Output

Your MPI program does not need to handle reading input from `stdin`. This will be handled in `common.cpp`. For output, we have provided a function `reportResult`. If you choose not to use this helper function, please adhere to the output format strictly as we will use the output to verify the correctness of your program.

2.2.1 Input Format

The input file is formatted as follows:

- The first line contains three integers: `num_data num_queries num_attrs` - number of datapoints, number of queries and the number of attributes each datapoint has.
- The next `num_data` lines each contain an integer label followed by `num_attrs` floating-point attributes. Each datapoint will automatically be 0-indexed in the order they are in.
- The final `num_queries` lines each begin with `Q`, followed by an integer `k` satisfying `1 <= k <= num_data`, and then `num_attrs` floating-point query attributes. Similarly, each query is automatically 0-indexed in the order they are in.

2.2.2 Output Format

Running `make` should produce two executables - `engine` and `engine.debug`. When running `engine`, the program should:

- Print one checksum line to `stdout` for each query, in the format
  `Query <query_id> checksum: <checksum>`.
  This is what the provided `reportResult` helper in `common.cpp` does.
- Print a single line to `stderr` containing the total time your program took, in the format
  `Time taken: <ms> ms`.

In `DEBUG` mode, it should **print for each query**:

- The most common label among the k nearest neighbors - in the case of tiebreaks, choose the label with the largest value

- The value k.
- Each neighbor's index and distance value with the **closest** point printed first - in the case of tiebreaks, choose the point with the larger index.
- The queries should be printed in order they appear in the input file.

Do not modify the output format, as it will be used to verify correctness. Disable or remove any additional debug or logging messages before submission.

2.2.3 Example

Sample Input

```
5 2 3
0 1.0 2.0 3.0
1 4.0 5.0 6.0
0 7.0 8.0 9.0
2 1.5 2.5 3.5
1 4.5 5.5 6.5
Q 2 1.0 2.1 3.0
Q 3 7.2 8.0 9.1
```

In the above sample input, there are 5 datapoints and 2 queries each with 3 attributes. The datapoints are indexed as 0,1,2,3,4 with labels 0,1,0,2,1 respectively.

Sample DEBUG Output

```
Label for Query 0 :  2
Top-2 neighbors:
0 :  0.01
3 :  0.66
Label for Query 1 :  1
Top-3 neighbors:
2 :  0.05
4 :  20.3
1 :  28.85
```

In the above sample output, for the first query, the closest point is the 0-th point with distance $(1.0-1.0)^2+(2.1-2.0)^2+(3.0-3.0)^2=0.01$ and the second closest point is the 3-rd point. Label 0 and 2 occurs once each among the top-2 neighbors, hence we assign the larger label 2.

3 Skeleton Code

You are provided with a skeleton implementation to get started:

File name	Description
`common.cpp` `common.h`	(Do not modify) Common library; contains definitions and implementations for I/O.
`engine.cpp`	This file contains the function KNN which you should implement
`benchmarks/`	Executables for correctness checks and for comparison against the speed of your code.
`Makefile`	Default Makefile for compiling the code. Modify this as you make more source and header files.
`generate_input.py`	Script to help you generate testcases.
`inputs.zip`	Archive containing the example testcases under `inputs/`. Extract it before using `run_bench.sh`.
`outputs/`	Directory created by `run_bench.sh` to store benchmark and engine outputs.
`run_bench.sh`	Example script to run your program on Slurm against the existing benchmarks, on a specific machine type.

Table 1: List of provided files in the skeleton

4 Grading

You are advised to work in groups of two students for this assignment (but you are allowed to work independently as well). You are not required to have the same teammate as past assignments. You may discuss the assignment with others but in the case of plagiarism, both parties will be severely penalised. If you use external references, cite them or at least mention them in your report (what you referenced, where it came from, how much you referenced, etc.). *This also includes use of AI tools such as ChatGPT, Copilot, etc.* Please refer to our policy in the introductory lecture on AI tool usage.

The grades are divided as follows:

- 6 marks – your parallel implementations with OpenMPI, split into:
  - 4 marks – the *correctness* of your simulation and implementation, described in Section 4.1.
  - 2 marks – the *performance* of your simulation, described in Section 4.2.
- 4 marks – your report, described in Section 4.3.
- Up to 1 bonus marks, described in Section 4.4

4.1 Correctness Requirements

In this section, we define the correctness requirements for your program. These requirements are in *addition* to any other information presented earlier in this document.

Your implementations should:

- Be written in C or C++, and compile and run successfully with `mpicxx` on all PDC lab machines.
- Create two executables named `engine` and `engine.debug` when you run `make` in the top level of your submission.
- Be parallelized using OpenMPI *only* – you are not allowed to use any other parallelization libraries, frameworks, or even C++ threading libraries or synchronization like `std::mutex`. You are also not allowed to submit a fully-sequential simulation – your program must show speedup over a single-threaded execution of your program.
- Be tested and run via Slurm. We will launch MPI programs in the same style shown in `run_bench.sh`: Open MPI's default rank mapping together with `--bind-to hwthread`.

- Have no memory leaks or undefined behavior.
- Not use any external utilities or libraries (e.g., those not provided with the C/C++ environment) that are not otherwise approved by the teaching team. Such approvals will be listed publicly in the FAQ document mentioned later. If what you want is not listed, you may request permission via our contact details in Section 5.1. Note that we are unlikely to allow things that directly impact performance.
- Use the provided `common.h` and `common.cpp` files *without any modifications*, as we don't want you to focus on this. You may change other parts of our code skeleton assuming that all other requirements are fulfilled. Our utility scripts can all be modified freely.
- **Have the same output compared to the one generated by our implementation**
  - We will check your code on test cases provided to you, and a number of private test cases.
  - We may update the our implementation if we discover any issues with it w.r.t our requirements.
- **Remain correct when executed with any number of nodes, with two or more ranks**.
- Not have any race conditions, data races, deadlocks, or any other synchronization issues.

4.2 Performance Requirements

We provide a number of **benchmark executables** to compare your solution against. Each are named in the format `bench_i`, where i is an integer. The i value does not necessarily represent any specific ordering of the benchmarks. You will be assigned performance marks based on your performance against these reference benchmarks. We do not disclose the specific mark allocation per benchmark.

Performance Metric: We will compare the wall-clock runtime of your program against the wall-clock time of each of the benchmarks. For each run, we will run each program with the same input file and the same number of ranks. We will run each program a few times (unspecified) and take the average (mean) time as the final time. We will launch MPI programs in the same style shown in `run_bench.sh`: Open MPI's default rank mapping together with `--bind-to hwthread`. **Important: we will run the benchmarks and your program on the same nodes (not just the same partition / type of node).**

Requirements: To receive marks for a specific benchmark, you must be faster than it under these conditions:

- Your program will run with n MPI ranks, where n is the total number of hardware threads across the allocated nodes for that benchmark configuration. If a node has P and E cores, we will only use the P cores.
- Your program will be run via Slurm.
- We reserve the right to deduct performance marks if there are correctness issues with your code.
- 80% of your performance grade will be from these specific executables, testcases, and machine types:
  - `bench_1`; `inputs/input1.in`; i7-7700×1 and i7-13700×1; 24 ranks total
  - `bench_2`; `inputs/input2.in`; i7-7700×1 and w5-3423×1; 32 ranks total
  - `bench_3`; `inputs/input2.in`; i7-13700×2; 32 ranks total
  - `bench_4`; `inputs/input3.in`; i7-13700×1 and w5-3423×1; 40 ranks total

Important: Each pair of machines in the PDC cluster may have different network speeds when communicating. That is, you might observe large differences among the interconnect speed between nodes in our PDC cluster. We will run the benchmarks and your program on the same nodes during grading.

Note that the scores are not distributed uniformly across these combinations, i.e. some configurations will have greater weight than others.

- 20% of your performance grade will come from our hidden testcases. The input will have these guarantees to ensure a high-enough workload, and therefore can benefit from parallelism:
  - num_data $\times$ num_attr $+$ num_queries $\times$ num_attr > 1000000
  - $k > 25$

We have tested our benchmark implementations, but as always, they are possibly incorrect. If you notice any issues, please do let us know. Our contact details are available in Section 5.1.

Slurm Usage

We reiterate that you should test and run your programs via Slurm. We will run your programs via Slurm, and if your program does not run correctly via Slurm, you will not receive marks. Furthermore, the CS3210 cluster will be heavily utilized during the assignment period, so Slurm allows a fair allocation of testing resources. **Do not test your program on login nodes!**

"Spirit of the Assignment" Note

We included a number of rules and constraints in this assignment for clarity. However, this is not a legal document – we cannot and do not want to cover every single eventuality in writing, and we also have certain learning objectives for you, so bypassing the spirit of the rules is not what we're aiming for. In particular, for this assignment, we encourage you to utilize all MPI ranks.

At the same time, we want to encourage exploration and interesting solutions. If you believe you have an interpretation of any these rules that is different from the one we might have intended, **please ask us for clarification** (even privately if necessary). For avoidance of doubt, if you have any concern that you are not interpreting the requirements as intended, you must ask us.

4.3 Report Requirements

4.3.1 Format

Your report should follow these specifications:

- Four pages maximum for main content (excluding appendix).
- All text in your report should be at least 11-point, in a readable typeface, and not be formatted in a way that is trying to bypass the page limit.

- All page margins (top, bottom, left, right) should be at least 1 inch (2.54cm).
- Have visually distinct headers for each content item in Section 4.3.2.
- It should be self-contained. If you write part of your report somewhere else and reference that in your submitted "report", we reserve the right to ignore any content outside the submitted document. An exception is referencing a document containing measurement data that you created as part of the assignment - we encourage you to do this.
- If headers, spacing or diagrams cause your report to *slightly* exceed the page limit, that's ok - we prefer well-organised, easily readable reports.

4.3.2 Content

Your report should contain:

- (1 mark) A brief description of your implementation, including:
  - An overview of your chosen algorithm, data structures, and parallelization strategy, and why these were chosen.
  - What OpenMPI constructs did you use, and why did you use those specific constructs.
  - How work is divided among ranks.
  - How you handled synchronization in your program.
  - How and why your program's performance scales with the number of ranks. Vary the number of ranks and present the data and trends clearly.

  Include any relevant details you think will help us understand your implementation.
- (2 marks) Description, visualization, and data on your execution, including:
  - How and why your program's performance changes based on parameters in the input file.
  - How and why your program's performance is affected by the type of machine you run it on. Include a comparison of at least *three* different hardware types.
- (1 marks) Describe at least ONE performance optimizations you tried, including supporting measurements and hypotheses on why they worked or did not work.

Please make sure to support your statements with clear data and graphs. We are looking for good quality, scientifically sound reports.

Additionally, your report should have an appendix (does not count towards page limit) containing:

- Details on exactly how to reproduce your results, e.g. nodes, inputs, execution time measurement, etc.
- Relevant performance measurements, if you don't want to link to an external document.

Tips:

- There could be many variables that contribute to performance, and studying every combination could be highly impractical and time-consuming. **You will be graded more on the quality of your investigations, not so much on the quantity of things tried** or even whether your hypothesis turned out to be correct.
- Performance analysis may take longer than expected and/or run into unexpected obstacles (like your program failing halfway). **Start early** and test selectively.

4.4 Bonus

You may obtain up to 1 bonus marks for achieving some of the fastest implementations in the class. More details can be found in the Assignment 3 Bonus section of the Student Guide (may take some time to be upa published)

5 Admin

5.1 FAQ

Frequently asked questions (FAQ) received from students for this assignment will be answered in the FAQ document here. The most recent questions will be added at the beginning of the file, preceded by the date label. **Check this file before asking your questions**.
If there are any questions regarding the assignment, use the Discussion section on Canvas.

5.2 Deadline and Submission

Assignment submission is due on **Friday, 17 April, 2pm**.

- **Canvas Quiz Assignment 3**: Take the quiz and provide the name of your repository and the commit hash corresponding to the `a3-submission` tag; if you are working in a team, only one team member needs to submit the quiz. If both of you submit, we will take the latest submission.
- **GitHub Classroom**: The implementation and report must be submitted through your GitHub Classroom repository. The GitHub Classroom for Assignment 3 can be accessed through https://classroom.github.com/a/2iy2MHCo.

The GitHub Classroom submission must adhere to the following guidelines:

- Name your team `a3-a0123456` (if you work by yourself) or `a3-a0123456-a0173456` (if you work with another student) – substitute your NUSNET number accordingly.
- Push your **code and report** to your team's GitHub Classroom repository.
- Your report must be a PDF named `<teamname>.pdf`. For example, `a3-a0123456-a0173456.pdf`. `<teamname>` should **exactly match** your team's name; if you are working in a pair, please DO NOT flip the order of your NUSNET numbers.
- Tag the commit that you want us to grade with `a3-submission`; if no such tag exists, we will use the most recent commit.

A penalty of 5% per day (out of your grade for this assignment) will be applied for late submissions.

Final check: Ensure that you have submitted to both **Canvas** and **GitHub Classroom**. Canvas should contain your commit hash and repository name, and GitHub Classroom should contain your code and report.